

Security & Compliance

Project name — TBD

Owner: seceng agent **Status:** *Draft* **Traces to:** PRD, ARCH **Last updated:** —

CONTENTS

1. Compliance Posture
2. Trust Boundaries
3. Threat Model (STRIDE)
4. Authentication & Authorization
5. Data Classification & Handling
6. Secret Management
7. Controls Catalog
8. CI/CD Security
9. Logging, Monitoring, Detection
10. Incident Response
11. Open Risks (Accepted)

1. Compliance Posture

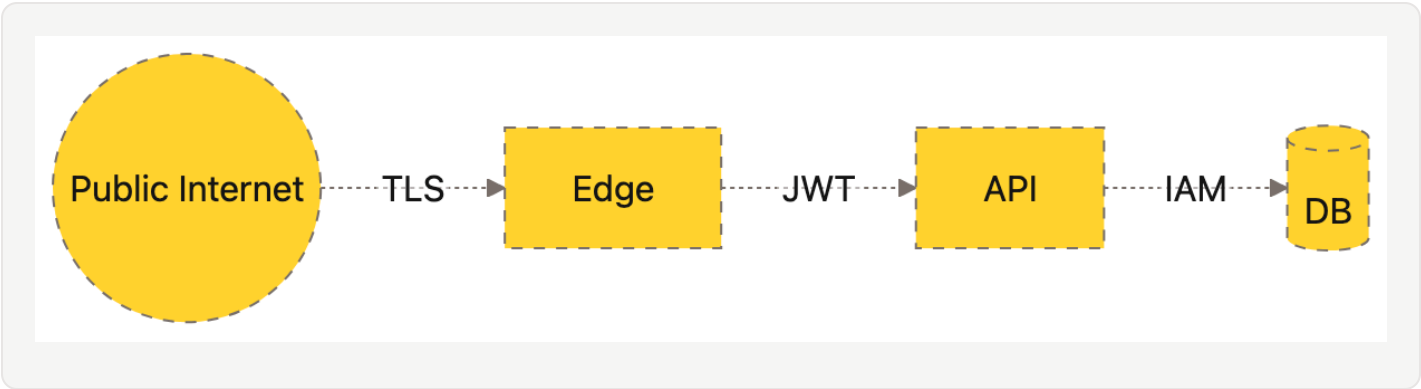
Source of truth is the PRD's Non-Functional Requirements section.

Regime	In scope?	Why	Target evidence
HIPAA	—	—	—
PCI-DSS	—	—	—
GDPR	—	—	—
SOC 2	—	—	—
Other	—	—	—

2. Trust Boundaries

Pulled from ARCH components. Any boundary becomes a threat-model entry.

From	To	Boundary type	Notes
—	—	—	—



3. Threat Model (STRIDE)

One table per component from ARCH. Focus on threats with non-trivial likelihood × impact.

Example — Component: *<name>*

Threat	STRIDE	Likelihood	Impact	Control(s)
—	—	—	—	—

4. Authentication & Authorization

How users and services authenticate; how authz decisions are made and enforced.

- **User authn:** — *provider, MFA policy, session model*
- **Service authn:** — *mTLS, signed JWTs, mesh identity*
- **Authorization model:** — *RBAC / ABAC / policy engine*
- **Enforcement point:** — *gateway / service / both*

5. Data Classification & Handling

Data type	Classification	At rest	In transit	Retention
—	—	—	—	—

6. Secret Management

- **Vault:** —
- **Injection:** — *how secrets reach runtime*
- **Rotation:** — *cadence, automation*
- **Access:** — *least privilege; who/what can read*
- **Forbidden:** secrets in code, secrets in logs, secrets in error messages, secrets in env files committed to git.

7. Controls Catalog

One row per control. Each maps to a threat-model entry.

Control	Threat addressed	Lives at	Tested by	Status
—	—	—	—	—

8. CI/CD Security

- Signed commits / tags: —
- Dependency scanning: —
- Container / image scanning: —
- SAST / DAST: —
- SBOM generation: —

9. Logging, Monitoring & Detection

- **Logged:** request ID, user ID, action, outcome. *Never:* raw secrets, full PII.
- **Storage & retention:** —
- **Alerts:** — *auth failure spikes, authz denials, anomalous egress*
- **Audit log:** — *sensitive operations, immutable storage*

10. Incident Response

- **On-call:** —
- **Severity criteria:** — *SEV1/2/3 definitions*
- **Containment → eradication → recovery:** — *runbook link*
- **Post-incident:** blameless postmortem within 5 business days; entry in `process/STATE.md` Decision Log.

11. Open Risks (Accepted)

Risks we live with — for now. Each needs an owner, compensating controls, and a re-evaluation date.

Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
Cross-root read access (PT-3): cairn serve reads directories named in a committed roots: config (or --repos), i.e. outside the repo. A malicious roots: entry in a PR could point a reviewer's local board at a sibling directory (e.g. ../../.ssh).	Read-only and localhost-only — no write path to a secondary root, and the /api/board payload carries only parsed cairn frontmatter, so there is no exfiltration channel. A directory without a config.yml is skipped before anything is read. Whoever can commit a roots: entry already has a stronger vector (editing cairn.py / board.js directly), so a runtime path-jail would be theater against the real threat.	Startup stderr banner logs each resolved absolute root path when >1 root loads (human tripwire against a stale/fat-fingered roots: silently serving the wrong tracker). Re-evaluate if cairn ever gains a write path to non-primary roots.	2027-02-21	Rich Mosko